

Veille Technologique — PKM Automatisé

Corentin Godon — Mastere 1 Expert IT, Développement & BDD — Epreuve E1

1. Présentation

Système de PKM (Personal Knowledge Management) automatisé qui collecte, score, resume et stocke des articles techniques sur le développement web (frontend, backend, securite). Accessible via un bot Discord 24h/24 avec des commandes slash. Deployé sur Fly.io (Paris).

2. Pipeline complet

Etape	Detail
1. Collecte (toutes les 2h)	Fetch RSS sequentiel — 3 articles max / source
2. Filtrage seen_urls	Articles deja vus ignores (table seen_urls SQLite)
3. Scoring Mistral AI	Score 1-5 de pertinence — article marque 'vu' dans tous les cas
4. Seuil >= 4	Articles sous le seuil ignores definitivement
5. Resume Mistral AI	Resume 3-4 phrases en francais (retry si rate limit)
6. Validation Discord	Message avec score affiche (score X/5) + boutons Valider / Rejeter
7a. Si valide	Insertion SQLite + post dans le salon thematique Discord
7b. Si rejete	Article ignore definitivement
8. Consultation	Commandes slash Discord disponibles

3. Routing automatique par theme

Quand un article est valide, il est automatiquement poste dans le bon salon Discord selon sa source :

Salon	Sources
#veille-frontend	Dev.to · Smashing Magazine · This Week in React · web.dev · Alsacreations
#veille-backend	Node Weekly · JavaScript Weekly · TypeScript Blog · Bun Blog · GitHub Changelog
#veille-securite	The Hacker News · CERT-FR (ANSSI) · Zero Day Initiative

4. Commandes Discord

Veille

/summary	5 derniers articles valides (titre, source, resume, lien)
/ask [question]	RAG : recherche SQLite + fallback Tavily web + synthese Mistral
/export	Export Markdown de toute la veille — envoye en piece jointe
/analyze	Analyse auto : tendances, sources actives, lacunes, recommandations
/submit [url]	Fetch la page, Mistral resume, envoye en validation Discord

Gestion des flux RSS

<code>/rss-list</code>	Liste les sources RSS actives avec leur ID
<code>/rss-add [nom] [url]</code>	Ajoute une source (initialise les defaults si premiere fois)
<code>/rss-remove [id]</code>	Supprime une source par son ID

Systeme

<code>/infos</code>	Statut : derniere collecte, prochaine collecte, nb articles en base
<code>/help</code>	Liste toutes les commandes (message visible uniquement par toi)

5. Stack technique

Composant	Technologie
Runtime	Node.js 20 + TypeScript
Collecte RSS	rss-parser (sources SQLite ou default)
IA scoring + resumes + analyse	Mistral AI (mistral-small-latest)
IA recherche web	Tavily Search API
Base de donnees	SQLite via better-sqlite3
Bot Discord	discord.js (commandes slash)
Hebergement	Fly.io — region Paris (cdg)
Versionning	GitHub (repo prive)

6. Base de donnees SQLite

Table	Role
articles	Articles valides (titre, url, source, resume, contenu, date)
seen_urls	URLs deja traitees — evite les doublons entre collectes
rss_sources	Sources RSS gerees dynamiquement via <code>/rss-add</code> / <code>/rss-remove</code>

7. Sources RSS par default (13 sources)

Axe	Sources
Frontend & Frameworks JS	Dev.to · Smashing Magazine · This Week in React · web.dev · Alsacreations
Backend, API & Outillage	Node Weekly · JavaScript Weekly · TypeScript Blog · Bun Blog · GitHub Changelog
Securite Web	The Hacker News · CERT-FR (ANSSI) · Zero Day Initiative

Collecte toutes les 2h — 3 articles max par source — delai 1s entre sources (anti-rate-limit). Gestion dynamique via `/rss-add` et `/rss-remove`. Les sources par default s'appliquent si aucune source n'est en base.

8. Variables d'environnement

Variable	Role
MISTRAL_API_KEY	Cle API Mistral AI
DISCORD_BOT_TOKEN	Token du bot Discord

DISCORD_CLIENT_ID	Application ID Discord
DISCORD_CHANNEL_ID	Salon principal (validation + commandes)
DISCORD_CHANNEL_FRONTEND	Salon #veille-frontend
DISCORD_CHANNEL_BACKEND	Salon #veille-backend
DISCORD_CHANNEL_SECURITE	Salon #veille-securite
TAVILY_API_KEY	Cle API Tavily Search (fallback web /ask)
DB_PATH	Chemin SQLite (./data/veille.db local, /data/veille.db Fly)

9. Infrastructure Fly.io

Parametre	Valeur
App	veille-tech-corentin
Region	cdg (Paris Charles de Gaulle)
VM	1 OCPU, 512 MB RAM, shared-cpu-1x
Volume persistant	veille_data — 1 GB, monte sur /data
Base SQLite	/data/veille.db
Image Docker	node:20-alpine (~193 MB)

Commandes Fly.io

<code>flyctl deploy --app veille-tech-corentin</code>	Deployer une nouvelle version
<code>flyctl logs --app veille-tech-corentin</code>	Logs en temps reel
<code>flyctl secrets set CLE=val --app veille-tech-corentin</code>	Mettre a jour un secret
<code>flyctl machines list --app veille-tech-corentin</code>	Lister les machines

10. Utilisation avec Claude pour la note de veille E1

1. /export sur Discord → recevoir veille_export_YYYY-MM-DD.md
2. Uploader ce fichier + context.md sur Google Drive
3. Ouvrir Claude (claude.ai) avec Google Drive connecte
4. Demander a Claude d'analyser et rediger la note de veille E1